Trucos evasivos de malware: Cómo un malware evade la detección por sandboxes



Autor: Clemens Kolbitsch
Fecha de Publicación: 1 marzo 2018
English

Los sandboxes son ampliamente utilizados para detectar malware. Proporcionan un entorno temporal, aislado y seguro para observar si un archivo sospechoso intenta algo malicioso. Por supuesto, los delincuentes

malware detecta una zona de pruebas, no ejecutará su verdadero comportamiento malicioso y, por lo tanto, parece ser otro archivo benigno. Si todo va bien desde el punto de vista del delincuente, el arenero liberará el archivo, lo entregará al usuario previsto y el malware puede lanzar el ataque contra el entorno real del usuario.

Es un juego del gato y el ratón en el que los proveedores de la zona de pruebas añaden nuevas técnicas para detectar malware y los delincuentes desarrollan formas creativas para evadir la detección y responder a las nuevas técnicas de detección añadidas al arenero.

Este artículo describe una muestra representativa de las técnicas que utilizan los delincuentes para evadir la detección, incluidos los métodos desarrollados más recientemente.[1]

Esto no pretende ser exhaustivo, especialmente teniendo en cuenta que continuamente se crean nuevas técnicas de evasión. Con cada revisión de sandbox, los delincuentes responden con una nueva técnica de evasión.

Este artículo proporciona ejemplos específicos de tres tipos de técnicas de evasión:

- **Evasión basada en el comportamiento del usuario**—Se usa para detectar acciones del usuario que indican la presencia de un usuario real o la inacción que indica una zona de pruebas. Los ejemplos de evasión basada en el comportamiento del usuario incluyen el uso de Application.RecentFiles.Count y la activación del código de macro al cerrar.
- **Evasión basada en máquina virtual (VM)**—Se usa para detectar artefactos que son indicativos de un entorno limitado basado en VM. Los ejemplos de evasión basada en VM incluyen la búsqueda de evasiones basadas en Zone:Identifier y Windows Management Instrumentation (WMI).
- **Evasión basada en el tiempo**—Se usa para evadir los areneros al retrasar la ejecución de un comportamiento malicioso o al detectar artefactos de temporización del espacio aislado. Los ejemplos de

y bombas de tiempo.

Ejemplos de evasión basada en el comportamiento del usuario

Los delincuentes despliegan una gama de técnicas para detectar la actividad del usuario que, según suponen, no estaría presente en una caja de arena. Dos de los ejemplos más recientes de esto son el uso de Application.RecentFiles.Count y desencadenar código malicioso cuando un documento está cerrado.

Use Application.RecentFiles.Count

Un gotero de malware Dridex reciente (malware que está diseñado para posteriormente instalar malware adicional) se distribuyó como un archivo de documento que contiene macros. Como antecedentes, Dridex es conocido como Bugat y Cridex (una forma de malware que se especializa en robar credenciales bancarias a través de un sistema que utiliza macros de Microsoft Word). Las macros usan la aplicación. RecentFiles.Count para verificar cuántos archivos se han accedido recientemente. Una cuenta baja sugiere que no hay una persona que usa la máquina y, por lo tanto, es más probable que la máquina sea una caja de arena.

Desencadenar macrocódigo en cierre

Los primeros sandboxes hicieron poco para emular la actividad del usuario más allá de abrir un archivo dentro de Microsoft Office. Como resultado, solo el código registrado para el evento Document_Open se desencadenaría dentro de la zona de pruebas. Sin embargo, los usuarios reales suelen interactuar con un documento mucho más. Se desplazan mientras leen, y una vez que terminan, cierran el documento. Se puede observar esta discrepancia entre el comportamiento de un usuario real y un entorno limitado, y el malware a menudo activa su código a través del evento Document_Close, lo que significa que solo ejecutará el código una vez que se cierre el documento.

Ejemplos de evasión basados en VM

Además de buscar la actividad del usuario, los delincuentes programan su malware para detectar cuándo se está ejecutando en una máquina virtual y, por lo tanto, es probable que sea un sandbox. Al igual que con la actividad de los usuarios, existe una larga lista de técnicas que utilizan los delincuentes, cuyos ejemplos detectados más recientemente se describen aquí.

Busqueda de zone: Identifier
Cuando se descarga un archivo desde Internet a una computadora que ejecuta Microsoft Windows, el sistema operativo agrega una secuencia de datos alternativa (ADS) al archivo para almacenar la Zone:Identifier del identificador. Estos metadatos incluyen información sobre el archivo, como información sobre la URL desde la que se descargó el archivo, y Windows la usa para mostrar los mensajes de advertencia apropiados al usuario antes de abrir contenido potencialmente no confiable.

Por otro lado, cuando un archivo se copia en un entorno limitado para su análisis, este metadato Zone:Identifier generalmente no está presente, ya que el entorno limitado no puede saber dónde se originó el archivo. El malware verificará esta discrepancia. La presencia de Zone:Identifier ADS da pistas sobre una máquina de usuario real. Si no se encuentra, el malware concluye que está en una caja de arena.

Evasiones basadas en WMI
La interfaz WMI permite que las máquinas con Microsoft Windows y cualquier servicio que se ejecute en ellas consulten información sobre procesos en ejecución, servicios disponibles, información de hardware (por ejemplo, disco) y más. Normalmente, los administradores del sistema usan WMI para automatizar tareas.

Como mínimo, las cajas de arena deben supervisar el tema principal, es decir, el programa que se va a ejecutar y los procesos con los que interactúa.

otro tipo de comunicación entre procesos (IPC), pero utiliza un modelo de cliente-servidor más complicado. Más precisamente, usa llamadas de procedimientos locales avanzadas (ALPC) para enviar consultas para ser ejecutado en el contexto de los procesos del servidor del sistema.

Si un sandbox no puede interceptar este tipo de comunicación, perderá las actividades realizadas por el malware utilizando WMI. Entre los ejemplos de malware que utilizan WMI para evadir los sandboxes se incluyen:

- **Verificación del recuento de núcleos**—Debido a las limitaciones de recursos, los sandboxes atribuyen los núcleos de la unidad de procesamiento central (CPU) mínimos necesarios a una VM, normalmente solo uno, por lo que pueden ejecutarse en paralelo en tantas VM en un servidor como sea posible. Sin embargo, la mayoría de las computadoras modernas tienen múltiples núcleos de CPU. El malware ejecutará una consulta WMI para recuperar el recuento de los núcleos, y si el valor es uno, concluye que se está ejecutando dentro de un recinto de seguridad.
- **Comprobación de espacio en disco y memoria física**—Al igual que en el caso de los núcleos de CPU, a las máquinas virtuales normalmente se les asigna una cantidad limitada de espacio en disco y memoria física. Para detectar si se está ejecutando en una máquina virtual, el malware comprueba si el espacio total en disco de la unidad es bajo, como por ejemplo debajo de 80 GB. De forma similar, verifica si hay una pequeña cantidad de memoria física, como menos de 1 GB de RAM. Estas configuraciones no suelen encontrarse en las máquinas de los usuarios finales.

Sin la capacidad de ver este tipo de IPC, un sandbox no puede interceptar (y manipular) los datos devueltos por el proceso del servidor. Por lo tanto, el malware encuentra los recursos de hardware limitados y detecta el entorno limitado.

Información de BIOS

La información básica del sistema de entrada/salida (BIOS) para máquinas virtuales y emuladores es diferente de la información del BIOS

encontradas en la información del BIOS para máquinas virtuales y puede verificar si la información actual del BIOS del sistema contiene esas cadenas. Si es así, el malware puede estar bastante seguro de que se está ejecutando en una máquina virtual.

Lista negra de geolocalización

Internet ofrece varios servicios que permiten a un usuario solicitar datos de geolocalización basados ??en la dirección IP del cliente. Maxmind es uno de esos servicios, y el malware puede consultar este servicio para obtener información sobre el sistema en el que se está ejecutando. Una parte de la información disponible es la empresa a la que se asigna el IP. El software malicioso compara estos datos con una lista de proveedores conocidos, por ejemplo, compañías de seguridad. Una coincidencia indicará que se está ejecutando dentro de una zona de pruebas.

Verificar nombre de usuario

El malware también toma las huellas dactilares del sandbox usando el nombre del usuario que ha iniciado sesión. Este truco funciona porque algunos proveedores no aleatorizan al usuario de Windows bajo el cual se ejecuta el análisis. El malware simplemente verifica el nombre de usuario contra una lista de nombres de usuario conocidos atribuidos a sandboxes. Por ejemplo, las versiones anteriores de dos conocidos sandboxes públicos, Hybrid Analysis y Malwr.com, solían tener nombres de usuario fijos, KR3T y PSPUBWS, respectivamente. Esto facilita que el malware detecte estas cajas de arena según el nombre del usuario actual.

Usando instrucciones específicas

Las tecnologías de virtualización modernas admiten instrucciones que provocarán incondicionalmente una “salida de VM” en el hipervisor (un sistema que crea y ejecuta máquinas virtuales). Esto permite que una máquina virtual modifique cómo se comporta la instrucción que desencadena la salida VM, de forma similar a un manejador de interrupciones. Sin embargo, esta interrupción introduce una discrepancia en el tiempo de ejecución.

Cuando se ejecutan en una máquina real, dichas instrucciones son más

detectar el hipervisor, lo que indica que se está ejecutando dentro de una máquina virtual. Por ejemplo, puede medir el tiempo de ejecución de la instrucción CPUID y compararla con el tiempo de ejecución esperado de esta instrucción en una máquina real.

Ejemplos de evasión basada en el tiempo

Una última categoría de evasión incluye técnicas que utilizan una variedad de mecanismos de sincronización. Los ejemplos detectados recientemente incluyen el uso de API de retraso, parche de sueño y bombas de tiempo.

Uso de API de retraso
Algunas cajas de arena están programadas para simplemente esperar y mirar durante un período de tiempo, y si un archivo no hace nada malicioso, liberará el archivo. Para evitar esto, el malware utiliza las API Sleep y NtDelayExecution disponibles en Windows. El malware llama a estas funciones para que duerman durante un período de tiempo para aguardar el sandbox.

Parches para dormir
Sandboxes parcheará la función dormir para tratar de superar el malware que utiliza retrasos de tiempo. En respuesta, el malware verificará si se aceleró el tiempo. El malware obtendrá la marca de tiempo, se irá a dormir y nuevamente obtendrá la marca de tiempo cuando se despierte. La diferencia de tiempo entre las marcas de tiempo debe ser la misma duración que la cantidad de tiempo que el programa malicioso fue programado para dormir. De lo contrario, el malware sabe que se está ejecutando en un entorno que está aplicando parches a la función de suspensión, lo que solo ocurriría en un entorno limitado.

Bombas de tiempo
Otra forma en que el malware intenta superar a los sandboxes es incluir código que solo se ejecutará en una fecha específica en el futuro (los

introducida por un sandbox.

Técnicas recomendadas para detectar malware

Para ser eficaces, las tecnologías de seguridad deben ser capaces de detectar malware que utiliza estas y muchas otras técnicas para evitar la detección, incluidas las nuevas estrategias de evasión que los delincuentes siguen desarrollando en respuesta a los sistemas de seguridad en constante mejora.

La buena noticia es que, si bien el malware evasivo plantea un desafío para las cajas de arena tradicionales, los entornos limitados de análisis modernos se basan en una técnica llamada emulación de sistema completo. La diferencia clave entre los sandbox basados en máquinas virtuales y los basados en emuladores de código es que las VM generalmente no virtualizan completamente la CPU utilizada para ejecutar código de malware. En cambio, pasa la mayoría de las instrucciones al hipervisor soportado por hardware subyacente para su ejecución.

Un emulador de código, por otro lado, maneja directamente cada instrucción ejecutada dentro del sistema de análisis y así puede manipular la ejecución de cualquier forma que el sistema desee. Esto se hace de una manera que es completamente transparente e invisible para el programa de malware bajo análisis.

Por ejemplo, un emulador de sistema completo puede alterar el resultado de las instrucciones de comparación de cadenas (por ejemplo, cuando se usa para comparar el nombre de usuario del sistema) y forzar la ejecución por una ruta que revela la intención verdadera de un programa. Del mismo modo, puede detectar cuándo un programa está ejecutando instrucciones que permiten tomar las huellas dactilares de la configuración del hardware y manipular el efecto de este código de una manera que desencadena un comportamiento adicional, ayudándolo a clasificar correctamente el

zona de pruebas una visibilidad completa del funcionamiento interno de los programas que se ejecutan dentro del entorno limitado de análisis. Es decir, en lugar de observar solo cómo un programa interactúa con el sistema operativo (por ejemplo, a través de llamadas al sistema), un emulador de código también puede rastrear datos procesados por las instrucciones que componen el programa de malware. Como resultado, el sandbox no solo puede rastrear qué tipo de datos se leen del sistema operativo, sino también cómo se usan, a qué valores se comparan (por ejemplo, en la toma de huellas dactilares del código del sistema), a dónde se envían los datos. (cuando se filtran datos confidenciales) y mucho más.

Por último, pero no menos importante, al tener visibilidad a nivel de instrucción en los programas bajo análisis, un emulador de código también puede razonar sobre las rutas de código que el programa de malware no ejecutó en una ejecución de análisis en particular. Por ejemplo, el sistema puede ver qué otro comportamiento potencial puede estar oculto en el malware que no se desencadenó durante el análisis dinámico, dando al sandbox incluso más información para clasificar una pieza de malware.

Conclusión

Los delincuentes son motivados, creativos y persistentes. Por cada mejora de sandbox utilizada para detectar malware evasivo, los delincuentes desarrollarán una técnica para evitar ser detectados y, a menudo, utilizarán múltiples técnicas en combinación para mejorar su éxito con la detección de un arenero. Las compañías de seguridad deben ofrecer, y sus clientes deben implementar, tecnologías avanzadas de detección de malware que sean efectivas independientemente de quién haya hecho el movimiento más reciente: el gato o el ratón.

Notas finales

[1] Lastline, *An Introduction to Advanced Malware and How It Avoids Detection*, 2017

Clemens Kolbitsch
Dirige el grupo antimalware en Lastline y trabaja en varios proyectos relacionados con el análisis y la detección. Como investigador de seguridad y desarrollador principal de Anubis, ha adquirido una gran experiencia en el análisis de códigos maliciosos actuales encontrados en la naturaleza. Ha observado varias tendencias en la comunidad de malware y publicado con éxito artículos de investigación revisados por pares. En el pasado, también investigó tecnologías ofensivas y presentó resultados en conferencias como BlackHat.

< PREVIOUS ARTICLE

NEXT ARTICLE >

Contact Us

Terms

Privacy

Cookie Notice

Cookie Settings

Fraud Reporting